biohttp

A call returns a value. It never raises.

Samuel Bharti

The soup this replaces

```
fetch_gene <- function(symbol) {
  url <- paste0("https://mygene.info/v3/query?q=", symbol)
  tryCatch(
    {
      body <- jsonlite::fromJSON(url)
      if (length(body$hits) == 0) return(NULL)
      body$hits[1, ]
    },
    error = function(e) NULL
  )
}
```

no such gene

`NULL`

network down

`NULL`

HTTP 503

`NULL`

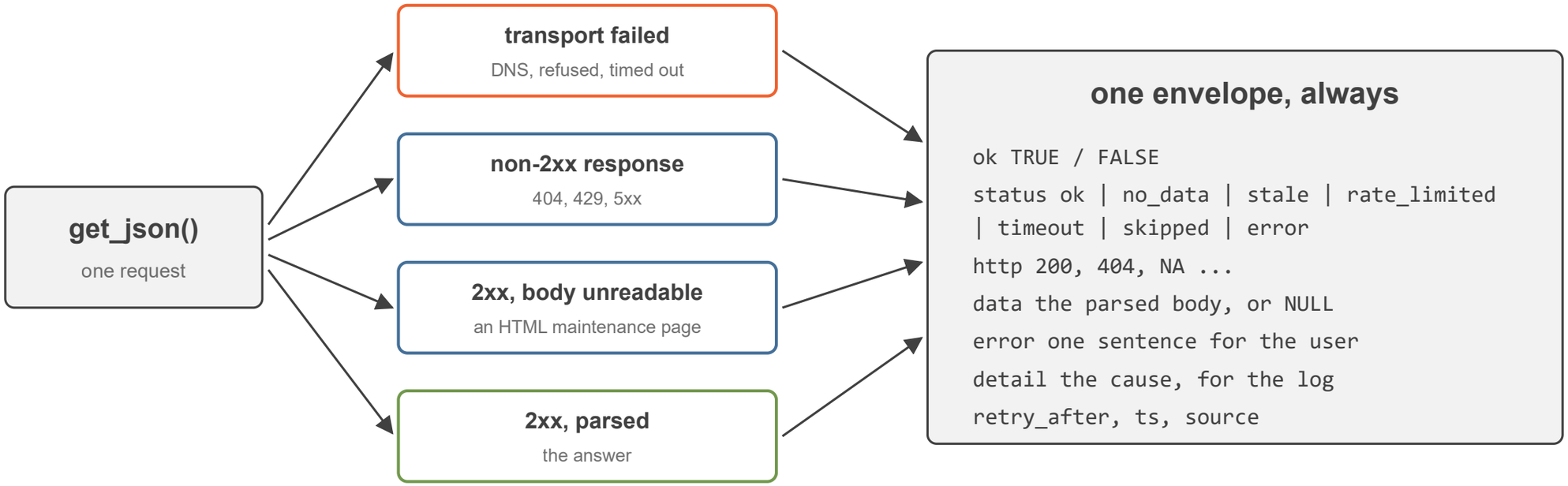
body was HTML

`NULL`

NULL erases why. A cache of NULLs poisons itself.

The one rule

A call returns a value. It never raises.



only the last outcome is cached; only the first counts against the circuit breaker

Nine fields

field	what it holds
`ok`	TRUE only when status is `ok`. Derived.
`status`	one of seven. What you branch on.
`http`	the code, or NA when nothing arrived
`data`	the parsed body on success
`error`	one sentence for the user
`detail`	the technical cause, for the log
`retry_after`	seconds the service asked you to wait
`source`, `ts`	who answered, when

`error` goes on screen. `detail` goes in the log. Never the other way round.

Seven statuses

ok | no_data | stale | rate_limited | timeout | skipped | error

`no_data`

An answer, not a fault. The source was reached and has nothing.

`skipped`

Nothing was sent. The host is paused. Evidence about the host, not the query.

`stale`

Real data, past its freshness window. `ok` is FALSE so it never gets cached.

Three of the seven are the ones a hand rolled client collapses into “failed”.

Branching

```
switch(res$status,
  ok           = render_table(res$data),
  no_data      = "Nothing found for that query.",
  skipped      = "Source paused, try again shortly.",
  rate_limited = paste("Slow down, retry in", res$retry_after, "s"),
  res$error
)
```

handle what you can act on
ok, no_data, skipped, rate_limited →

let the rest fall through
`res$error` is already a sentence →

`body_or_null()`
When you honestly do not care why

Every branch is testable offline with one mocked response.

The breaker rule

> “Only a transport failure counts against a host. Any HTTP response at all, including a 500 and including a 200 whose body will not parse, proves the host is reachable and clears the count.”
>
> from the biohttp vignette

3
FAILURES TO OPEN

120
SECONDS COOLDOWN

0
REQUESTS WHILE OPEN

A 503 proves the host is up. Taking it out of rotation turns a partial outage into a total one.

Success-only cache

Stored

- a 2xx that parsed

Never stored

- timeout, refused, 5xx
- `no_data` from a 404
- `stale`, `skipped`

Failures are fetched again every time.

variable	default
`BIOHTTP_CACHE_TTL`	1800 s
`BIOHTTP_CACHE_DISK`	off
`BIOHTTP_CACHE_DIR`	`R_user_dir()`
`BIOHTTP_CACHE_DISK_TTL`	7 days
`BIOHTTP_CACHE_SALT`	empty

Per call: `ttl = 86400` for a bulk table.

Many questions, one host

```
transport_stats_reset()
res <- get_json_many(
  "https://mygene.info/v3", path = "query",
  queries = lapply(c("BRCA1", "TP53", "EGFR"), \(g) list(q = g)),
  source = "MyGene",
  throttle = list(capacity = 10, fill_time_s = 60)
)
transport_stats()
#>          host dispatched retried rate_limited
#>   mygene.info          3       0            0
```

serve from cache
what is already held

→

send the rest
grouped by host, throttled

→

retry only the retryable
429, timeout, 5xx; honour Retry-After

→

`transport_stats()`
what actually went out

Results come back in the order you asked. Zip them onto your input by position.

Testing: mocks, and one real server

`httr2::with_mocked_responses()`

Every status but one. One response object per test. Runs anywhere.

`statuses, branching, cache rules`

`webfakes`

Retry needs a real socket, because mocks sit below the retry loop.

`retry, timeout, Retry-After, breaker`

```
httr2::with_mocked_responses(
  list(httr2::response(status_code = 503)),
  expect_identical(mygene_query("BRCA1")$status, "error")
)
```

Worth asserting: a failure is never cached, a 5xx never trips the breaker, a token never prints.

Start here

CRAN 0.1.2

```
install.packages("biohttp")
```

r-universe 0.1.3

```
install.packages("biohttp",
  repos = "https://samuelbharti.r-universe.dev")
```

0.1.3 adds `retry_after`, batched retry, `ratelimit_*()`, `transport_stats()`.

Write a client in six lines

```
mygene_query <- function(symbol) {
  res <- get_json("https://mygene.info/v3",
    path = "query", query = list(q = symbol),
    source = "MyGene")
  if (!res$ok) return(res)
  status_ok(pluck_at(res$data, "hits"), "MyGene")
}
```

No Shiny dependency, on purpose. The app is the caller.